



BANNARI AMMAN INSTITUTE OF TECHNOLOGY

An Autonomous Institution Affiliated to Anna University Chennai - Approved by AICTE - Accredited by NAAC with "A+" Grade

SATHYAMANGALAM - 638 401 ERODE DISTRICT TAMIL NADU INDIA

Ph: 04295-226000 / 221289 Fax: 04295-226666 E-mail: stayahead@bitsathy.ac.in Web: www.bitsathy.ac.in

22AI503/22AM503 MACHINE LEARNING

UNIT II – DATA PREPROCESSING

Data Transformation and Data Discretization

1. Introduction

1.1 Definition

Data transformation converts data into forms appropriate for mining or machine learning. Data discretization converts continuous numeric attributes into discrete intervals (bins or categories), making them suitable for algorithms that require categorical input or reducing the number of distinct values.

Definition:

Data Transformation: Converting data into appropriate formats — normalisation, aggregation, generalisation, smoothing, and feature engineering.

Data Discretization: Partitioning continuous numeric values into a finite set of intervals or categories. Both steps are essential preprocessing stages that make raw data compatible with ML algorithm assumptions.

1.2 Importance in Machine Learning

Many ML algorithms have strict requirements: linear models assume normally distributed features, distance-based algorithms require features on the same scale, naive Bayes requires categorical inputs, decision trees handle discretized data more efficiently. Transformation and discretization bridge the gap between raw data and algorithm requirements. Incorrect scaling causes kNN and SVM to be dominated by high-magnitude features. Untransformed skewed features cause linear regression to produce biased coefficients.

1.3 Role in Data Preprocessing Pipeline

Transformation and discretization are typically the final preprocessing steps before modelling. They are applied after cleaning, integration, and reduction. The complete preprocessing pipeline flows: Raw Data → Attribute Profiling → Cleaning → Integration → Reduction → Transformation → Discretization → Model Training.

2. Learning Outcomes

Understand

- Five transformation types: normalisation, aggregation, generalisation, smoothing, attribute construction
- Normalisation methods: Min-Max, Z-score standardisation, Decimal Scaling
- Discretization methods: equal-width, equal-frequency, entropy-based, cluster-based

Apply

- Implement Min-Max and Z-score normalisation using sklearn.preprocessing
- Apply Box-Cox and Yeo-Johnson power transformations to skewed distributions
- Implement equal-width, equal-frequency, and entropy-based binning

Analyze

- Select the appropriate normalisation method based on algorithm and data distribution
- Evaluate information loss in different discretization strategies
- Choose between top-down (splitting) and bottom-up (merging) discretization approaches

3. Detailed Technical Concepts — Data Transformation

Figure 6.1 – Data Transformation Methods Overview

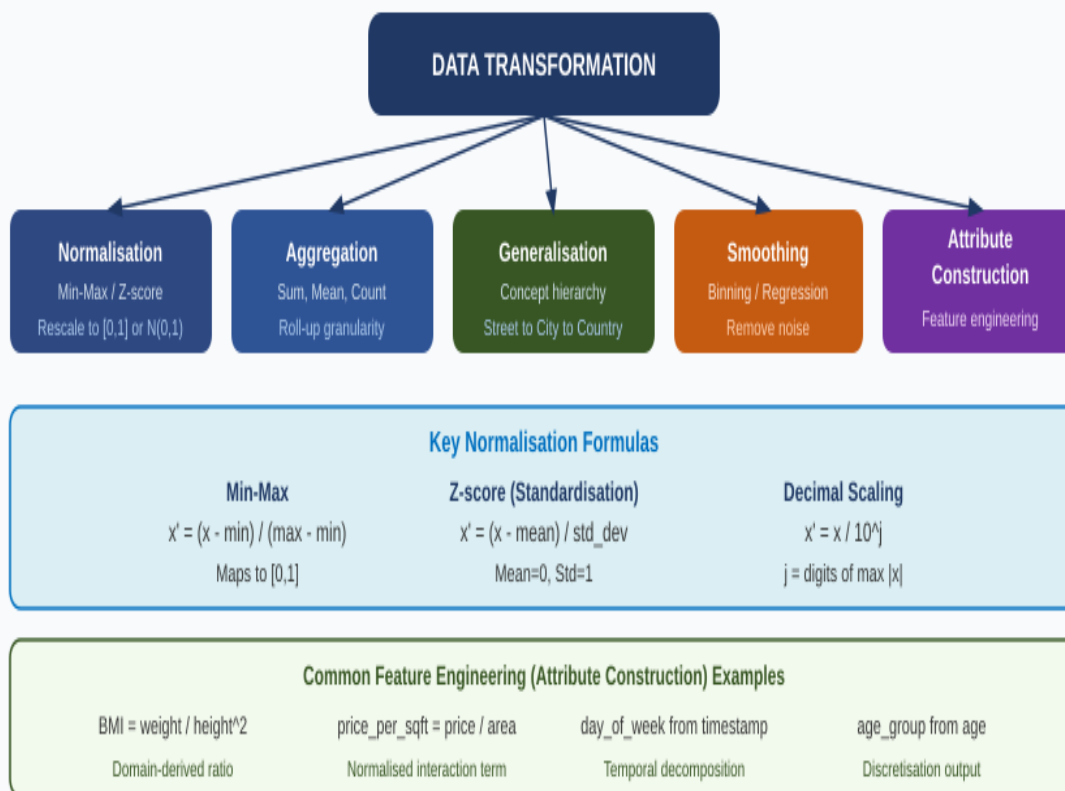


Figure 6.1 – Data Transformation Methods Overview with Key Formulas

3.1 Normalisation / Scaling

Normalisation rescales attribute values to a standard range or distribution. Essential for distance-based algorithms (kNN, k-Means, SVM), gradient-descent optimisers (neural networks), and regularised models (Ridge, LASSO).

3.1.1 Min-Max Normalisation

$$x_{\text{new}} = (x - x_{\text{min}}) / (x_{\text{max}} - x_{\text{min}})$$

Maps all values to the range [0, 1]. Preserves relative relationships and zero values. Sensitive to outliers — if a single outlier has an extreme max value, all other values get compressed near 0.

Example: Income values [20K, 50K, 80K, 100K, 500K]. Max=500K. Min-Max maps 20K→0, 100K→0.16, 500K→1. The outlier 500K compresses the rest into [0, 0.16].

sklearn: `MinMaxScaler(feature_range=(0,1)).fit_transform(X)`

Variant: `MinMaxScaler(feature_range=(-1, 1))` maps to [-1, 1] — useful for tanh activation functions.

3.1.2 Z-Score Standardisation

$x_{\text{new}} = (x - \text{mean}) / \text{std_dev}$

Produces a distribution with mean=0 and standard deviation=1. Robust to outliers compared to Min-Max. Recommended when data approximately follows a normal distribution. The output is in units of standard deviations from the mean.

When to use Z-score over Min-Max: when the algorithm assumes Gaussian-distributed input (logistic regression, LDA, PCA), when outliers are present but must be kept, when the range of the feature is unknown at deployment time.

sklearn: `StandardScaler().fit_transform(X)`

3.1.3 Decimal Scaling

$x_{\text{new}} = x / 10^j$ where j is the smallest integer such that $\max(|x_{\text{new}}|) < 1$.

Example: values in range [-900, 800] → $j=3$ → divide by 1000 → range [-0.9, 0.8]. Simple integer scaling. Rarely used in modern ML pipelines; superseded by Min-Max and Z-score.

3.1.4 Robust Scaler

$x_{\text{new}} = (x - \text{median}) / \text{IQR}$. Uses median and IQR instead of mean and std. Highly robust to outliers — IQR is not influenced by extreme values. Recommended when data has many outliers that should not be removed.

sklearn: `RobustScaler().fit_transform(X)`

3.2 Power Transformations (Handling Skewness)

Power transformations reduce skewness to make distributions closer to normal — a requirement for linear models, LDA, and probabilistic models.

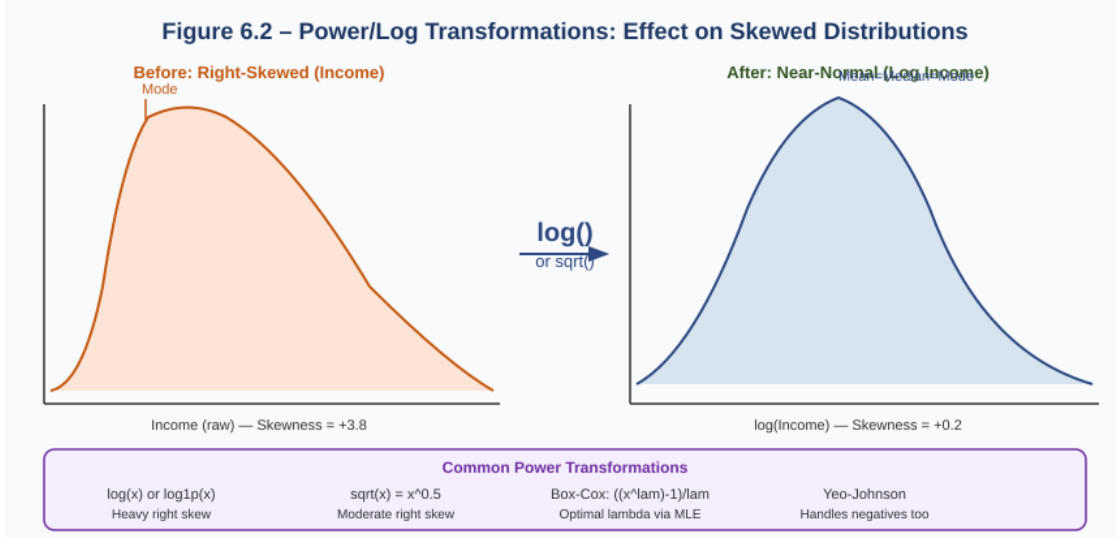


Figure 6.2 – Log and Power Transformations Effect on Right-Skewed Distribution

3.2.1 Log Transformation

$x_{\text{new}} = \log(x)$ or $\log(x + 1)$ for zero values ($\log1p$). Best for heavy right-skewed distributions (income, house prices, count data). Compresses large values while spreading small values.

Requirement: $x > 0$. For zeros: use $\log1p(x) = \log(1+x)$. For negatives: use Yeo-Johnson transformation.

3.2.2 Square Root Transformation

$x_{\text{new}} = \sqrt{x}$. Less aggressive than log. Suitable for moderate right skew. Also requires $x \geq 0$.

3.2.3 Box-Cox Transformation

$x_{\text{new}} = (x^\lambda - 1) / \lambda$ if $\lambda \neq 0$, or $\log(x)$ if $\lambda = 0$.

Lambda is chosen to maximise the log-likelihood of a normal distribution fit. $\lambda=1$ (no transform), $\lambda=0$ (log), $\lambda=0.5$ (sqrt), $\lambda=-1$ (reciprocal). Requires strictly positive values ($x > 0$).
sklearn: `PowerTransformer(method="box-cox")`.

3.2.4 Yeo-Johnson Transformation

Extension of Box-Cox that handles zero and negative values. Applies different formulas for positive and negative x depending on λ . sklearn: `PowerTransformer(method="yeo-johnson")` — recommended over Box-Cox for general use.

3.3 Aggregation

Aggregation combines multiple values into a single value by rolling up granularity. Common in data warehousing and time-series preprocessing.

- Examples: Daily sales → Monthly sales (sum). Individual customer transactions → Customer lifetime value (sum or mean). City-level data → Country-level data (mean or total).
- pandas: `df.groupby("month")["sales"].sum()`; `df.resample("M")["value"].mean()` for time-series.
- Trade-off: Aggregation reduces data volume and noise but loses fine-grained detail. Choose aggregation level based on the prediction target granularity.

3.4 Generalisation (Concept Hierarchy Climbing)

Generalisation replaces low-level attribute values with higher-level concepts in a predefined concept hierarchy. This reduces dimensionality and increases abstraction.

Attribute	Low Level (Raw)	Mid Level	High Level	Highest Level
Location	Street address	ZIP Code / PIN	City	Country
Date/Time	2024-01-15 14:32:05	Jan 2024, 14h	2024 Q1	2024
Age	27 years	25-34 age group	Young Adult	Adult
Product	iPhone 15 Pro 256GB	iPhone 15 series	Smartphone	Electronics
Temperature	98.6 F / 37.0 C	Normal body temp	Healthy	Normal

3.5 Attribute Construction (Feature Engineering)

New attributes are constructed from existing ones using domain knowledge. This is one of the most impactful preprocessing steps — often called "feature engineering."

- Ratio features: BMI = weight_kg / height_m^2; price_per_sqft = price / area
- Temporal features: extract year, month, day_of_week, hour, is_weekend from timestamp
- Interaction features: age × income, distance × speed = travel_time
- Difference features: account_balance_change = current_balance - previous_balance
- Polynomial features: x^2 , x^3 , $x*y$ for capturing non-linear relationships in linear models

4. Detailed Technical Concepts — Data Discretization

Discretization converts a continuous numeric attribute into a nominal/ordinal one by partitioning the value range into intervals. This is required for algorithms like Naive Bayes (assumes discrete inputs), Apriori association rule mining, and decision trees that prefer discrete splits.

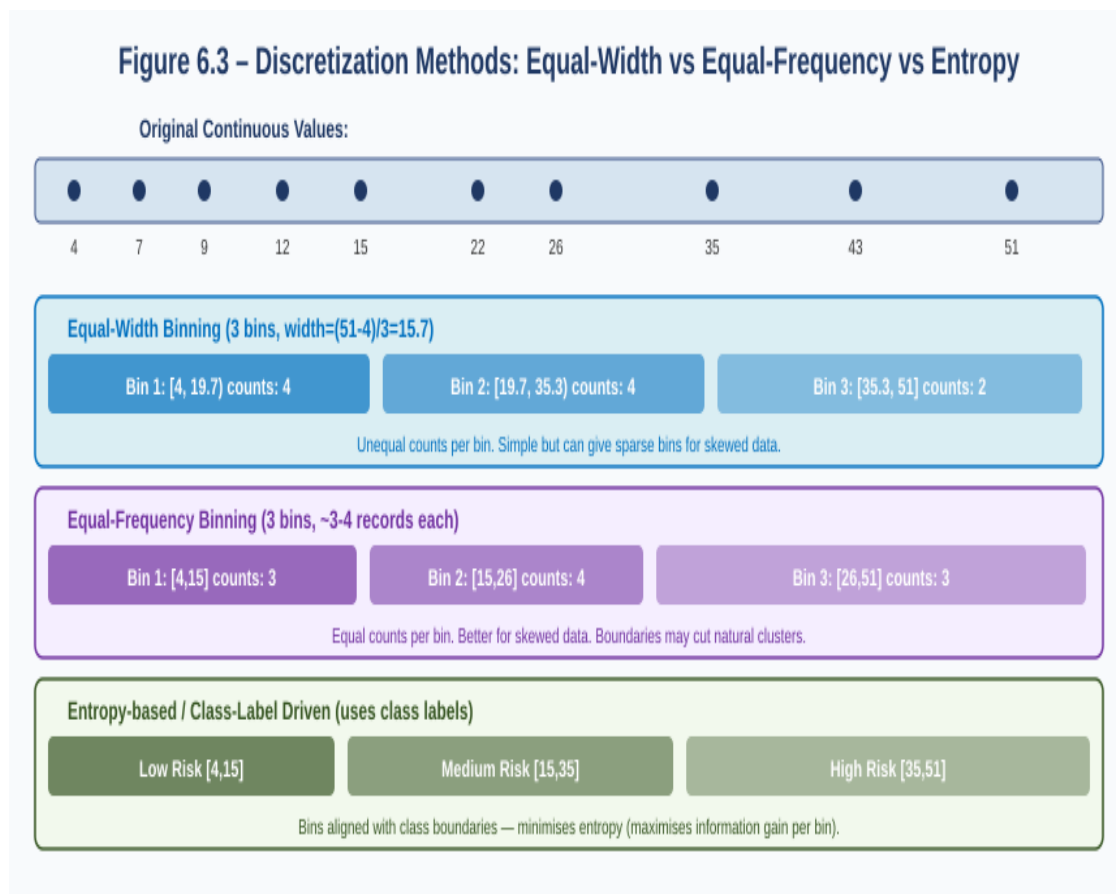


Figure 6.3 – Discretization Methods: Equal-Width, Equal-Frequency, Entropy-based

4.1 Unsupervised Discretization

4.1.1 Equal-Width Binning (Equi-Width)

Divides the range [min, max] into k bins of equal width: width = (max - min) / k. Simple and fast. Disadvantage: can create bins with very few or no records for skewed distributions.

Example: Age range [0, 100], k=5 bins → [0-20), [20-40), [40-60), [60-80), [80-100]

pandas: `pd.cut(df["age"], bins=5)` — equal-width by default

4.1.2 Equal-Frequency Binning (Equi-Depth / Quantile)

Each bin contains approximately the same number of records. Bin boundaries are set at quantile points (25th, 50th, 75th percentile for 4 bins). Better handles skewed data. Bin widths are unequal.

pandas: `pd.qcut(df["income"], q=4, labels=["Low", "Med-Low", "Med-High", "High"])`

4.1.3 Cluster-Based Discretization

Apply 1D k-Means clustering to the attribute values. Cluster centroids define natural groupings. Labels are assigned based on cluster membership. Captures natural density-based groupings in the data that uniform binning misses.

4.2 Supervised Discretization

4.2.1 Entropy-Based (Top-Down Splitting)

Uses class labels to find split points that maximise information gain. The algorithm evaluates every possible split point, computes the entropy of resulting bins, and selects the split with minimum weighted entropy (maximum information gain).

Information Gain: $IG(S,T) = Entropy(S) - \sum_{\text{over bins}} (|S_i|/|S| \times Entropy(S_i))$

Where $Entropy(S) = -\sum (p_i \times \log_2(p_i))$ for each class proportion p_i .

This process is recursively applied to each sub-interval (like a 1D decision tree). Stopping criterion: minimum description length (MDL) principle or minimum bin size.

4.2.2 ChiMerge (Bottom-Up Merging)

Starts with each unique value as its own bin. Iteratively merges adjacent bins that are most similar in class distribution (lowest chi-squared statistic). Merging stops when all adjacent pairs have chi-squared above a threshold (significant class difference).

Chi-squared between adjacent bins i and j : $\chi^2 = \sum (O_{ij} - E_{ij})^2 / E_{ij}$ for each class. Low chi-squared $\rightarrow$ similar class distribution $\rightarrow$ safe to merge.

4.3 Choosing the Right Discretization Method

Method	Type	When to Use	Limitation
Equal-Width	Unsupervised	Uniform data without outliers	Poor on skewed data; empty bins possible
Equal-Frequency	Unsupervised	Skewed data; NB and rule-based models	May cut natural clusters at quantile boundary
Cluster-based	Unsupervised	Data with natural density groupings	Sensitive to k choice; non-convex clusters
Entropy (ID3)	Supervised	Classification tasks with class labels	Computationally expensive; can overfit
ChiMerge	Supervised	Bottom-up merging; fewer assumptions	Slow for large k initial bins

4.4 Concept Hierarchies in Discretization

User-defined concept hierarchies explicitly map continuous values to meaningful categories based on domain knowledge. Example: Loan-to-Value ratio for mortgage:

- $LTV < 0.50$: "Very Low Risk" (equity-rich)
- 0.50 to 0.80 : "Standard" (conventional mortgage)
- 0.80 to 0.95 : "High Risk" (PMI required)
- $LTV > 0.95$: "Very High Risk" (likely default)

Domain-driven discretization outperforms statistical binning when domain boundaries are well-established.

5. Industry Tools and Technologies

5.1 Commercial Tools

Tool	Vendor	Transformation Features	Discretization Features
IBM SPSS Modeler	IBM	Normalise, derive, aggregate nodes	Binning node: equal-width, optimal binning
SAS Enterprise Miner	SAS	Box-Cox, normalisation, roll-up	Variable clustering, entropy bins
Alteryx Designer	Alteryx	Formula tool, multi-row formulas	Tile tool: equal records/width
Talend Studio	Talend	300+ transform functions	tMap with conditional bin logic
Databricks	Databricks	PySpark MLlib feature transformers	Bucketizer, QuantileDiscretizer

5.2 Open-Source Tools

- sklearn.preprocessing: MinMaxScaler, StandardScaler, RobustScaler, PowerTransformer (Box-Cox, Yeo-Johnson), QuantileTransformer, Binarizer
- pandas: pd.cut() (equal-width), pd.qcut() (equal-frequency), cut labels and retbins parameters
- sklearn.preprocessing: KBinsDiscretizer(n_bins=5, encode="ordinal", strategy="uniform/quantile/kmeans")
- featuretools: Automated temporal feature extraction and aggregation transforms
- category_encoders: TargetEncoder, BinaryEncoder, WOEEncoder (Weight of Evidence) for high-cardinality categoricals
- scipy.stats: boxcox(), yeojohnson() for power transformations with lambda estimation

6. Real-World Applications

Domain	Transformation Applied	Discretization Applied	Benefit
Credit Scoring	Log(income); Z-score all numeric	Income bins: Low/Mid/High/Premium	Improved LR model compliance with normality
Healthcare	Age normalised; BMI derived from ht/wt	BMI: Underweight/Normal/Overweight/Obese	Naive Bayes model for obesity risk
Retail/Marketing	RFM scores derived (Recency/Freq/Money)	RFM deciles for customer segments	Targeted campaigns per segment
Manufacturing	Sensor readings: RobustScaler	Defect level bins: OK/Warning/Critical	SVM anomaly detection on standardised data

Energy	Daily kWh: aggregated to monthly	Usage: Low/Medium/High/Peak	Time-series classification for demand forecasting
--------	----------------------------------	-----------------------------	---

7. Advantages and Limitations

Advantages	Limitations
Normalisation ensures all features contribute equally to distance metrics	Min-Max is sensitive to outliers — single extreme value compresses all others
Power transforms reduce skewness to satisfy model normality assumptions	Log transform requires strictly positive values; workaround adds complexity
Discretization enables naive Bayes and association rule mining on continuous data	Equal-width binning creates empty bins for sparse or skewed distributions
Feature engineering adds domain knowledge increasing model accuracy	Entropy-based discretization is supervised — cannot be used in unsupervised tasks
Aggregation reduces noise and smooths temporal fluctuations	Generalisation and aggregation cause irreversible information loss

8. Case Study

CASE STUDY

Case: Predicting Hospital Readmission — Transformation Pipeline

Problem: A hospital readmission prediction model had AUC=0.71 on raw features. Data science team suspected transformation issues after plotting feature distributions.

Dataset: 100,000 patient records | 30 numeric features | Target: 30-day readmission (binary)

Issues Identified via EDA:

- num_medications: right-skewed (skewness=+2.9)
- days_in_hospital: right-skewed (skewness=+1.8)
- time_in_hospital: approximately normal
- age: ordinal (10 age groups, not true numeric)
- num_lab_procedures: approximately uniform

Transformation Actions:

- Log1p transform: num_medications, days_in_hospital (skewness reduced to +0.2, +0.4)
- Z-score standardisation: all numeric features post-transformation
- Age: discretized into 4 groups: Young (<=30), Adult (31-60), Senior (61-80), Elderly (>80)
- New features: medication_density = num_medications / days_in_hospital (feature engineering)
- QuantileTransformer: applied to num_lab_procedures (uniform → approx. normal)

Results:

- Logistic Regression AUC: 0.71 → 0.79 (+8% from transformation alone)
- Feature medication_density ranked #1 in RF importance
- Age discretization improved Naive Bayes performance from 0.63 → 0.71

Conclusion: A systematic transformation pipeline — addressing skewness, scaling, discretization, and feature engineering — improved model performance by 8% without any algorithmic changes.

9. Scenario-Based Learning

SCENARIO 1

SCENARIO 1: SVM Fails Without Normalisation

Background: An ML team trains an SVM to classify e-mail spam. Features include: word_count (range 1-2000), avg_sentence_length (1-100), num_links (0-50), exclamation_count (0-500), currency_symbol_count (0-30).

Technical Environment: Python, sklearn SVM (RBF kernel), 50,000 emails.

Problem Statement: SVM with default settings achieves only 61% accuracy. Training takes 45 minutes.

Team suspects the model is not converging properly.

Available Evidence: df.describe() shows: word_count mean=320, std=180; exclamation_count mean=12, std=45. Feature scales vary by factor of 40x.

Investigation: SVM with RBF kernel uses Euclidean distance. word_count (0-2000) dominates the kernel computation; exclamation_count (0-500) has large variance but a smaller absolute scale compared to word_count.

Analysis: Apply StandardScaler. After Z-score: all features have mean=0, std=1. SVM decision boundary can now weigh all features appropriately.

Findings: After StandardScaler, SVM accuracy: 61% → 94%. Training time: 45 min → 3 min. The RBF kernel now operates correctly in a balanced feature space.

Recommended Solution: Always scale features before SVM, kNN, k-Means, PCA, or any distance-based algorithm. Use StandardScaler for normal-ish distributions, MinMaxScaler for bounded features, RobustScaler when outliers are present.

SCENARIO 2

SCENARIO 2: Entropy-based Binning for Fraud Risk Scoring

Background: A fintech company needs to classify transactions as Low Risk, Medium Risk, or High Risk based on transaction_amount (continuous, range \$1 to \$50,000).

Technical Environment: Python, sklearn KBinsDiscretizer, DecisionTree, pandas, 1M transaction records.

Problem Statement: Equal-width bins of transaction amount (\$0-16K, \$16K-33K, \$33K-50K) showed no useful class separation: fraud rate was 1.2%, 1.4%, 1.6% — barely different.

Available Evidence: Fraud rate by quantile: Q1 (\$0-500): 0.3% fraud; Q2 (\$500-2K): 1.1%; Q3 (\$2K-10K): 3.8%; Q4 (\$10K+): 8.9%.

Investigation: Use entropy-based discretization (supervised with fraud label). Identify split points that maximise information gain.

Analysis: Entropy-based splits found at: \$0-\$500 (Low Risk), \$500-\$5,000 (Medium Risk), \$5,000+ (High Risk). These align with fraud rate jumps (0.3% → 2.5% → 8.9%).

Findings: Naive Bayes model using entropy bins: precision=0.82, recall=0.77. Same model with equal-width bins: precision=0.44, recall=0.51. Entropy bins 2× better precision.

Recommended Solution: When class labels are available and the goal is supervised classification, always prefer entropy-based or ChiMerge discretization over equal-width/equal-frequency. Validate bin thresholds with domain experts.

SCENARIO 3

SCENARIO 3: Feature Engineering for Demand Forecasting

Background: A logistics company needs to forecast warehouse demand. Raw data: daily shipment counts with timestamps. Linear Regression on raw count predicts poorly (RMSE=1,450 units).

Technical Environment: Python, pandas, sklearn LinearRegression, XGBoost, 3 years of daily data.

Problem Statement: Raw daily count feature does not capture seasonality, day-of-week effects, or holiday impact. Model treats all days identically.

Available Evidence: EDA scatter of day_number vs shipments shows clear weekly cycles and annual peaks (Nov-Dec holiday surge). Mean shipments: Mon=2,300, Fri=4,100, Sat=800.

Investigation: Apply temporal feature engineering from the timestamp column.
 Analysis: Extract: day_of_week (0-6), month (1-12), is_weekend (binary), is_holiday (binary), quarter, week_of_year. Discretize count into bins: Low/Medium/High/Peak using quantiles.
 Findings: Linear Regression with engineered features: RMSE drops from 1,450 to 680 (53% improvement). XGBoost with same features: RMSE=312. day_of_week and is_holiday ranked as top-2 features by importance.
 Recommended Solution: Always extract rich temporal features from timestamps for time-series problems. Consider lag features (yesterday count, last_week_same_day), rolling means (7-day average), and cyclical encoding (sin/cos of month for periodicity).

10. Common Errors and Troubleshooting

Error	Cause	Impact	Fix
Fitting scaler on full dataset (train+test)	Data leakage: test statistics used in training	Overly optimistic test performance	Always fit scaler on training set only; apply (transform) to test
Applying Min-Max with outliers present	One extreme value compresses all others to ~0	kNN and SVM accuracy degrades	Use RobustScaler or clip outliers before Min-Max
Log transform on zero or negative values	$\log(0) = -\infty$	ValueError; model breaks	Use $\log_{1p}(x)$ for zeros; Yeo-Johnson for negatives
Equal-width bins on sparse skewed data	Most records fall in 1-2 bins; others empty	Naive Bayes fails on empty bins	Use equal-frequency (pd.qcut) for skewed distributions
Applying transformation inside cross-val loop incorrectly	Fitting transformer on all folds combined	Data leakage, inflated CV scores	Use sklearn Pipeline to ensure transformer is fit per fold

11. Key Terminologies

Term	Definition
Normalisation	Rescaling attribute values to a standard range [0,1] using Min-Max formula
Standardisation	Z-score transformation: $(x - \text{mean})/\text{std}$. Produces mean=0, std=1 distribution
Min-Max Scaling	$x_{\text{new}} = (x - \text{min})/(\text{max} - \text{min})$. Sensitive to outliers; maps to [0,1]
Z-score	$(x - \text{mean})/\text{std}$. Standardised score. $ z > 3$ flags outliers.
RobustScaler	Scaling using median and IQR. Robust to outliers.
Box-Cox	Power transformation $((x^\lambda - 1)/\lambda)$ to reduce skewness. Requires $x > 0$
Yeo-Johnson	Extension of Box-Cox handling zero and negative values
Log Transform	$x_{\text{new}} = \log(x)$. Compresses right-skewed distributions
Aggregation	Combining multiple values to higher granularity (sum, mean, count)
Generalisation	Climbing concept hierarchy: raw value to higher-level concept
Feature Engineering	Constructing new attributes from existing ones using domain knowledge
Discretization	Converting continuous numeric attribute into discrete bins/categories

Equal-Width Binning	Divides range into k bins of equal width. <code>pd.cut()</code>
Equal-Freq Binning	Each bin has equal count of records. <code>pd.qcut()</code>
Entropy-based Binning	Supervised: splits maximise information gain using class labels
ChiMerge	Bottom-up supervised discretization: merges adjacent bins with low chi-squared

12. Topic Summary

UNIT II – Topic 6

DATA TRANSFORMATION AND DISCRETIZATION — SUMMARY

- Transformation Types: (1) Normalisation/Scaling, (2) Aggregation, (3) Generalisation, (4) Smoothing, (5) Attribute Construction.
- Scaling Methods: Min-Max (range [0,1], sensitive to outliers), Z-score (mean=0, std=1, robust), RobustScaler (uses median/IQR, best with outliers).
- Power Transforms: log/log1p (heavy right skew), sqrt (moderate skew), Box-Cox (auto-optimised lambda, $x > 0$), Yeo-Johnson (handles negatives).
- CRITICAL: Always fit scalers/transformers on TRAINING SET ONLY. Apply (transform) on test set. Use sklearn Pipeline.
- Discretization Unsupervised: Equal-Width (`pd.cut` — uniform data), Equal-Frequency (`pd.qcut` — skewed data), Cluster-based (natural groupings).
- Discretization Supervised: Entropy-based top-down (maximises information gain), ChiMerge bottom-up (merges similar adjacent bins).
- Feature Engineering: Derive domain-meaningful features (BMI, price_per_sqft, temporal components) for significant accuracy gains.
- Generalisation: Street → City → Country concept hierarchies reduce cardinality and improve abstraction.
- Tools: sklearn (MinMaxScaler, StandardScaler, RobustScaler, PowerTransformer, KBinsDiscretizer), pandas (`pd.cut`, `pd.qcut`), scipy (boxcox, yeojohnson).